

Lab 05

Intro to Make and Environments

Thanks to STAT 159's Fall 2025 staff,
some of these slides are heavily
inspired by their material.

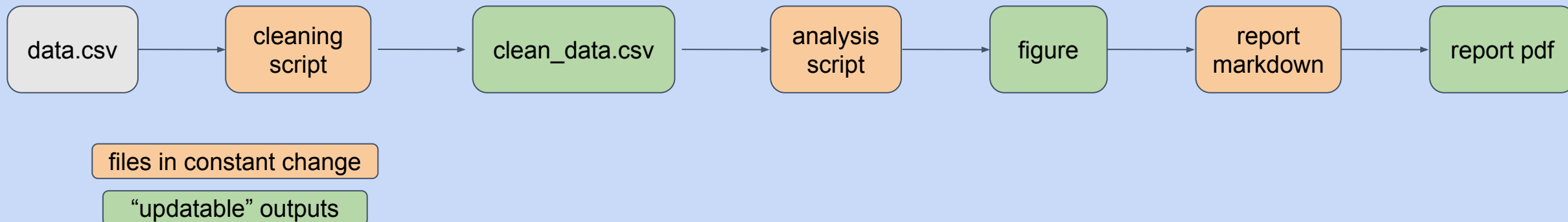
Today's plan

01	Bash scripts	A little bit more bash, but now in scripts
02	Makefiles	1 rule, several rules, PHONY targets
03	A first look into virtual environments	For R and Python
04	Reading Checkpoint #4	Reading Data Science at a Singularity

Automation

- So far we have run everything by hand: *file and folder creation, Python/R scripts and notebooks*
- As workflows grow and become more complex, steps depend on each other
- Manual = not reproducible, does not scale, easy to get wrong or to break due to different setups
- The idea:

define what files we need to “build” (known as *targets*) that depend on each other, run any of them reproducibly, and avoid repeating unnecessary work



Scripts context

Scripting languages are widely used nowadays for **data - stats - ML tasks, web development** and certain areas of **software development**.

- R, Python, PHP, Perl, Ruby, Bash are some examples of scripting languages
- They are interpreted, flexible, in general easier to learn, but generally slower than their “compiled” counterparts (C, C++, Java).
- We can run code in these languages either interactively, line by line (REPL, Jupyter notebooks), or by executing a saved script file (*python3 [script.py](#)* or *Rscript script.R*).

→ Follow along of R and Python script running.
DataHub > shared folder > bcourses... > lab05 > mini_scripts

Bash scripting - overview

We have used the terminal before as a way of communicating with the operating system.

Shell:

- primary program computers use to receive the commands and return outputs
- These commands can be entered and executed via the *terminal*.

Bash:

- (it comes from “Bourne Again SHell”)
- it’s an implementation of Shell,
- allows you to efficiently perform many tasks,
- e.g. you can use Bash to perform operations on multiple files quickly via the CLI.

Bash scripting - overview

You can also write and execute scripts in Bash, just like you can in R or Python, that can be executed across different OS.

Sometimes bash and shell are used interchangeably.

- A bash script is a text file of shell commands, run top to bottom
- Bash Shebang: first line `#!/bin/bash`
- Make it executable: `chmod +x script.sh`. Run it: `./script.sh`
- Variables and loops exist

Bash scripting - overview

Example of bash script:

```
#!/bin/bash  
name="make"  
echo "Hello from $name"  
  
for n in 1 2 3; do  
    echo "step $n"  
done
```

Bash scripting limitations

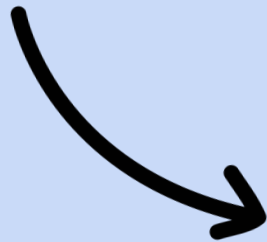
- It **always runs everything**, even when nothing changed
- It doesn't know which steps **depend on which files**
- As it grows, you keep track of what needs re-running

e.g. let's try running:

```
#!/bin/bash  
echo "Running python script"  
python3 hello.py  
echo "Running R script"  
Rscript greet.R Alice
```

Bash scripting limitations

- It **always runs everything**, even when nothing changed
- It doesn't know which steps **depend on which files**
- As it grows, you keep track of what needs re-running



Enter: **Make**

Make

- A tool used to build code from 1976, still widely used
- It is based on shell language
- You describe **targets**, their **dependencies** and the **recipe** (commands) to build them
- Make re-runs only what is out of date, using timestamps as a reference
- Newer tools (Snakemake, pydoit, SCons, Makeflow) are more sophisticated, but share these ideas



TARGET: DEPENDENCIES
COMMANDS

output.txt: Alice
python3 greet_on_steroids.py

Structure of a rule

- target: dependencies, then a recipe line that starts with a tab
- use # to comment
- The recipe is just bash
- If you indent with spaces instead of a tab will output: missing separator

```
# Compute prime numbers
results/report.md: clean_data.csv
    python analysis.py input1.txt results/output1.txt
```

How Make decides what to run

- Make compares modification times
- Target missing, or older than any dependency → run the recipe
- Otherwise: 'results/output1.txt' is up to date

Brief demo & try for yourself Lab Notebook #5 - Part 1

*but first, change to tab indentation in DataHub

Break

More than one rule, target or dependency

- More than one target: plain make builds only the first target (the default goal)
- More than one dependency: list the script too, so changing the code triggers a rebuild
- Dry run: make -n prints commands without running them

PHONY targets

- Some targets are not files: `clean`, `all`, `setup`, `help`
- If a file named `clean` exists, `make clean` says 'up to date' and does nothing
- `.PHONY:` `clean` declares that the target is just a name for commands
- The first target is the default: put `all` at the top

```
.PHONY: all
all: results/output1.txt results/output2.txt
```

```
.PHONY: clean
clean:
    rm -f results/*
```

Real examples of Makefiles

- <https://github.com/drivendataorg/cookiecutter-data-science>
- <https://github.com/scikit-learn/scikit-learn/tree/main>

Now try for yourself - Lab notebook #5 - Part 2

Virtual environments

Could you have the same code but different results?

- Code always runs in an environment: one version of Python/R, one set of libraries
- Different versions can give different results or errors
- To reproduce results we need to record the dependencies

What is an environment?

- An isolated set of installed packages, accessed by name
- You can have many (one per project) and switch between them
- Your project's packages don't interfere with others

Environment files

- They are like ingredient lists.
- A file that declares the environment: rebuild it anywhere
- Commit it to git next to your code
- Pin versions: python=3.13, not just python

environment.yml (Python, conda)

```
name: primes-env
channels:
  - conda-forge
dependencies:
  - python=3.13
  - numpy=2.3
```

Python and R envs

Same principle with different tools. conda works for both languages.

Idea	conda (Python or R)	Python only	R only
Isolated packages	conda environment	<code>python -m venv .venv</code>	<code>renv</code> project library
File that records them	<code>environment.yml</code>	<code>requirements.txt</code>	<code>renv.lock</code>
Rebuild from the file	<code>conda env create -f environment.yml</code>	<code>pip install -r requirements.txt</code>	<code>renv::restore()</code>
Record current state	<code>edit file / conda env export --from-history</code>	<code>pip freeze</code>	<code>renv::snapshot()</code>

Good practices

- One environment per project
- Always record packages in the file, not only with install at the command line
- Pin versions, commit the file
- Environments are disposable, so you can restart it (recreate it) with no problem if necessary

Now try for yourself - Lab notebook #5 - Part 3

Reading Checkpoint #4